

Part 4

APRENDE PYTHON

Errores y Archivos

APRENDE PYTHON

Errores y Archivos

Tu código **va a fallar**, y eso es normal. Aprenderás a **entender los errores**, a que no tumben tu programa, y a **leer y guardar datos** en archivos.

```
print(10 / 0)

# ZeroDivisionError:
# division by zero
```

💡 Un error no es un castigo: es Python diciéndote **qué** salió mal y **dónde**. Leerlos bien es una de las habilidades que más te harán crecer.

Errores | try / except | Leer archivos | Escribir archivos

Milagros Espinoza Cunya

APRENDE PYTHON

Errores y Archivos

1 Entender los errores

¿Qué es un error?

Un mensaje de Python cuando algo sale mal. Te dice el **tipo** de error y la **línea** donde ocurrió. No le temas: es tu mejor pista.

Los que más verás:

`SyntaxError`	Escribiste mal el código (falta un **:** o un paréntesis).
`NameError`	Usaste una variable que **no existe** o mal escrita.
`TypeError`	Mezclaste tipos: sumar **texto + número**.
`KeyError`	Buscaste una **clave** que no está en el diccionario.

Resultado:

```
>>> edad = "30" + 5
TypeError: can only concatenate
str (not "int") to str
```

👉 En simple: intentaste unir un texto (`"30"`) con un número (`5`), y Python no puede mezclarlos. Solución: conviértelos primero → `int("30") + 5` = 35.

DATAZO

Lee el error **de abajo hacia arriba**: la última línea te dice QUÉ pasó, y las de arriba DÓNDE. El 90% de las soluciones están en googlear esa última línea.

Milagros Espinoza Cunya

APRENDE PYTHON
Errores y Archivos

2 try / except

¿Qué hace?
Le dice a Python: "**intenta** esto; si falla, **no te caigas**, haz esto otro". Así tu programa sigue vivo aunque haya un error.

Problema
Si el usuario escribe texto donde esperamos un número, el programa se cae. Queremos evitarlo.

La Solución:

```
try:
    edad = int("hola")   # esto falla
except ValueError:
    print("Eso no es un número válido")
```

`try` → **intenta esto** `except` → **si falla, haz esto**

Resultado:

```
Eso no es un número válido   # siguió corriendo
```

DATAZO

Sin **try/except**, un solo dato malo tumba todo tu análisis. Con él, lo atrapas y sigues. Es lo que hace que un programa sea **robusto**.

Milagros Espinoza Cunya

APRENDE PYTHON
Errores y Archivos

3 Leer archivos

¿Para qué?
Casi nunca escribes los datos a mano: vienen en archivos (**.txt**, **.csv**). Python los abre, los lee y los trae a tu código.

La Solución:

```
with open("datos.txt") as archivo:
    contenido = archivo.read()
    print(contenido)
```

`open()` abre el archivo · `with` lo cierra solo al terminar · `.read()` lee su contenido

El modo por defecto es `"r"` (read, lectura). Con `with` no tienes que cerrar el archivo manualmente: se cierra solo.

Resultado:

```
Rubí,8.9
Teresa,9.0
Rebelde,9.8
```

DATAZO

Siempre usa **with open()**. Si abres un archivo y olvidas cerrarlo, puedes corromper datos o llenar la memoria. **with** te protege de eso solito.

Milagros Espinoza Cunya

APRENDE PYTHON
Errores y Archivos

4 Escribir archivos

¿Para qué?
Guardar tus resultados en un archivo para no perderlos: un reporte, una lista limpia, datos procesados.

La Solución:

```
with open("reporte.txt", "w") as f:
    f.write("Rebelde: Éxito\n")
    f.write("Teresa: Regular\n")
```

`"w"` write: crea/reemplaza | `"a"` append: agrega al final | `\n` salto de línea

Cuidado: el modo `"w"` borra lo que haya antes. Si quieres sumar sin borrar, usa `"a"` (append).

DATAZO

El `\n` al final de cada línea es clave: sin él, todo tu texto se escribe pegado en una sola línea gigante. Es el error más común al escribir archivos.

Milagros Espinoza Cunya

APRENDE PYTHON
Errores y Archivos

5 Archivos CSV

¿Qué es un CSV?
El formato de datos más usado del mundo: una tabla donde cada valor se separa con comas. Excel, bases de datos y Python lo entienden.

La Solución:

```
import csv

with open("novelas.csv") as f:
    lector = csv.reader(f)
    for fila in lector:
        print(fila)
```

`import` trae herramientas extra. `csv` ya viene con Python y te evita separar las comas a mano.

Resultado:

```
['titulo', 'rating']
['Rubí', '8.9']
['Rebelde', '9.8']
```

DATAZO

En la Parte 5 conocerás **Pandas**, que lee un CSV entero en **una sola línea** y lo convierte en una tabla lista para analizar. Pero entender esto primero te da la base.

Milagros Espinoza Cunya

APRENDE PYTHON

Errores y Archivos

Lo que te llevas de esta parte:

Para...	Usas	¿Qué hace?
Atrapar errores	`try / except`	Evita que un fallo tumbe tu programa.
Abrir archivo	`with open()`	Lo abre y lo cierra solo al terminar.
Leer	`.read()`	Trae el contenido a tu código.
Escribir	`.write()` + `"w"/"a"`	Guarda resultados en un archivo.
Datos en tabla	`import csv`	Lee archivos separados por comas.

¡Guarda este post, dale like, compártelo y sígueme para la Parte 5: Pandas!

Milagros Espinoza Cunya